

Stop vibe-coding. Start engineering with agents.

The habits that make senior engineers fast with AI — and why the "one giant prompt" era is over. Follow the line down; it walks you from the problem to a working way of shipping.

▼ FOLLOW THE NUMBERED LINE, TOP TO BOTTOM

1

THE MIRROR

You already use AI — just the **old way**.

Most of us drive agents like a vending machine: one big prompt, straight to code, one session for everything, and the reviewer catches the mess. It feels fast. It isn't.

OLD WAY

- ✗ One mega-prompt → implementation
- ✗ One session, many tasks
- ✗ Ship AI code without reading it
- ✗ Reviewer becomes the safety net



NEW WAY

- Grill → align → spec → thin slices
- A fresh session per task
- Self-review + tests before the MR
- Feedback loops catch it first

So why does one giant session go wrong? It comes down to how the model's memory works.

2

THE ONE IDEA

The **smart zone** is real.

Context is the agent's working memory. Reasoning is sharpest early; as the window fills, accuracy quietly degrades. This single fact explains almost every bad AI output you've seen.



The window is large, but quality slips well before it's full. Watch `/context`. The fix is never "power through" — it's a **new session**.

Staying in the smart zone is rule #1. Here are the six that hold everything up.

3

THE CORE

Six rules, **any level**.

Forget everything else and these six still carry you — beginner or advanced.

01

Context is working memory

Best reasoning early. When the window fills, start a fresh session — don't push on degraded thinking.

02

One session = one task

Cramming many tasks into one window is a one-way trip to the dumb zone.

03

Align before you implement

Grill and plan first. A one-shot mega-prompt skips the alignment that makes code correct.

04

AI accelerates entropy

Bad code fails *faster* with AI → tests, review & feedback loops matter **more**, not less.

05

Steer the agent

`CLAUDE.md` + skills make the right behaviour the default, not a hope.

06

Ship vertical slices

Thin end-to-end tracer bullets, validated early — not horizontal layers reviewed at the end.

Rules need mechanisms. The first lever is steering — telling the agent how we work.

4

STEERING

Teach the agent how we work.

Two tools do the steering: a lean `CLAUDE.md` that loads on every request, and skills that load only when relevant.

A Writing a good CLAUDE.md

It sits at the top of every request, so every token competes for attention. Keep the root tiny; push detail into files it references on demand.

```
# CLAUDE.md — keep this small
One-line: what this project is.
Package manager: pnpm (only if non-standard)
Build / test: pnpm verify

# load detail only when relevant:

@docs/git-workflow.md
@frontend/CLAUDE.md # nested, per-folder
@api/testing.md
```

✓ **Reference, don't inline.** `@relative/path` pulls a file in on demand — keeps the root lean.

✓ **Nest per folder.** `frontend/CLAUDE.md` loads when work happens there.

✓ **Keep** conventions, pitfalls & rationale that differ from tool defaults.

✓ **Cut** dir listings & dependency dumps — the agent reads those. `/init` to start, `/doctor` to trim.

B Skills — and writing your own

Skills are discoverable instructions the agent loads on demand — the right guidance at the right moment, without bloating context.

✓ **Two kinds.** *User-invoked* orchestrate (you type `/grill-me`); *model-invoked* hold reusable discipline the agent reaches for automatically.

✓ **Make one when** you keep pasting the same checklist, or a `CLAUDE.md` section has grown into a recipe.

There's a skill for writing skills. `/writing-great-skills` — the principles that make a skill predictable. `skill-creator` — scaffold, edit, and eval a skill's trigger accuracy.

Anyone can add a skill to the shared set — the on-ramp to the practice (see the end).

Steering makes the right thing likely. Feedback loops make it certain.

5

FEEDBACK & QUALITY

Give the agent a reality check.

An agent with no signal is flying blind. Fast, automated feedback is how good code stays good — exactly what the "old way" skips.

Feedback loops

Fastest first: types → tests → lint → review → QA. Each catches error before it compounds. Wire them so the agent runs them itself.

TDD · red→green→refactor

`/tdd` forces a failing test first, then the minimum to pass, then cleanup — one vertical slice at a time.

Entropy is the enemy

AI doesn't make bad code tolerable — it makes it **fail faster**. Deep modules + `/improve-codebase` keep it navigable.

Now assemble it. Here's the exact path a change travels — in order.

THE MAIN FLOW

6

Idea → ship, in order.

The numbered steps are the route most feature work takes. Never jump ahead of your own alignment.

1

`/grill-with-docs`

Reach shared understanding & build the domain model.

↳ no codebase → `/grill-me` · very complex → `/wayfinder` OPTIONAL

↳ need to dig deep? new session → `/research` → return here

2

`/prototype` FRONTEND

For UI/logic questions — build it small and validate before you spec.

3

`/to-spec`

Synthesize the thread into a PRD and publish it.

4

`/to-tickets`

PRD → independently-grabbable tracer-bullet tickets.

↳ keep grill → spec → tickets in ONE window; then **clear between tickets**

5

`/implement`

Fresh session per ticket · drives `/tdd` at agreed seams.

6

`/code-review` or `/iws-review`

Two-axis review — standards + spec — before you deliver.

Simple task or bug? Collapse the path: `/grill-me` → `/implement` → `/code-review` .

The toolbox behind the flow — and how to move between sessions.

THE TOOLBOX

7

Skills & session moves.

`ful` stateful (saves files) `less` stateless (nothing saved).

SKILL	STATE	USE
<code>/grill-me</code>	less	align — no codebase
<code>/grill-with-docs</code>	ful	align + domain model / ADRs
<code>/wayfinder</code>	ful	map huge work as tickets
<code>/research</code>	ful	dig deep vs primary sources
<code>/feature-deep-dive</code>	less	understand code first
<code>/prototype</code>	less	throwaway to validate
<code>/to-spec</code> · <code>/to-tickets</code>	ful	PRD → tracer-bullet tickets
<code>/implement</code> · <code>/tdd</code>	less	build one slice, red-green
<code>/code-review</code> · <code>/iws-review</code>	less	standards + spec, pre-deliver
<code>/writing-great-skills</code>	less	write skills well

Session moves

<code>/compact</code>	summarize in place — continues
<code>/handoff</code>	save to file — forks new session
<code>/clear</code>	wipe context — keeps memory

★ Show live context % in your terminal —

`/statusline` or `ccstatusline`. You can't stay in the smart zone if you can't see it.

Top commands

<code>/context</code>	what fills the window	<code>/rewind</code>	roll code+chat back
<code>/plan</code>	enter plan mode	<code>/model</code> · <code>/effort</code>	model / reasoning
<code>/init</code>	starter CLAUDE.md	<code>/memory</code>	edit memory
<code>/resume</code>	reopen a session	<code>/usage</code>	cost + limits

Keys: `Shift+Tab` cycle modes · `Esc Esc` rewind · `!` bash

You've got the practice. Here's how to set it up on your machine.

GET SET UP

8

Install the flow.

On a brownfield codebase the agent setup usually already exists — you don't write a fresh PRD; you start from feature PRDs and follow the CLAUDE.md + skills already in the repo. On anything new, one command sets up the whole toolkit.

Set up the skills, then shape them.

Adopt the flow, then help shape it: contribute a skill, sharpen your CLAUDE.md files, share what works. Better workflow, keeping pace with the field, and a better product.

```
# install the skill set
npx github:letStayFoolish/ai-flow-setup
```